

Smart contract audit raindex

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	5
Security findings	6
**** Critical	6
*** High	6
H01 - takeOrders4 delays calculate-phase state persistence, allowing repeated-order batches to bypass stateful order limits	6
** Medium	7
M01 - Vault ID 0 on Order Input Side Requires Pre-existing Ambient Balance on OrderBook	7
M02 - order-counterparty() is exposed in add/remove post-actions even though that context row is never populated	8
M03 - arb4 computes flash-loan size from minimumIO without respecting IOIsInput semantics	8
* Low	9
L01 - Rounding policy in pullTokens and pushTokens can trap residual dust in the orderbook	9
L02 - Deposit and withdraw post-action contexts include token decimals that the subparser does not expose	10
L03 - RouteProcessorOrderBookV6ArbOrderTaker bypasses TOFU decimal handling and is less compatible with non-standard tokens	10
Informational	11
I01 - onTakeOrders2 can be abused to drain accidental balances	11
I02 - takeOrders4 Push-Callback-Pull Pattern Exposes Partial State	12
General Risks and Assumptions	14
Approach and methodology	15



Project description

Raindex (rain.orderbook) is a decentralized, permissionless orderbook protocol built for deterministic financial execution on the EVM. This repository contains the core OrderBookV6 engine, specialized arbitrage-taker contracts (FlashBorrower and OrderTaker), and an integrated sub-parser for the Rain interpreter. The protocol is centered around a custom decimal floating-point math system that utilizes a 256-bit data structure—comprising a 224-bit signed coefficient and a 32-bit signed exponent—to ensure exact on-chain representation of human-readable values. By rejecting standard IEEE 754 binary precision and "nonsense" values (like NaN or Infinity) in favor of strict, deterministic errors, the system provides a high-fidelity environment for trustless, complex trading strategies.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/raindex

Reviews

Date	Repository	Commit
02/04/26	raindex	e686b4d353bfa271b1eef99c2569426eaa075b9b

Scope

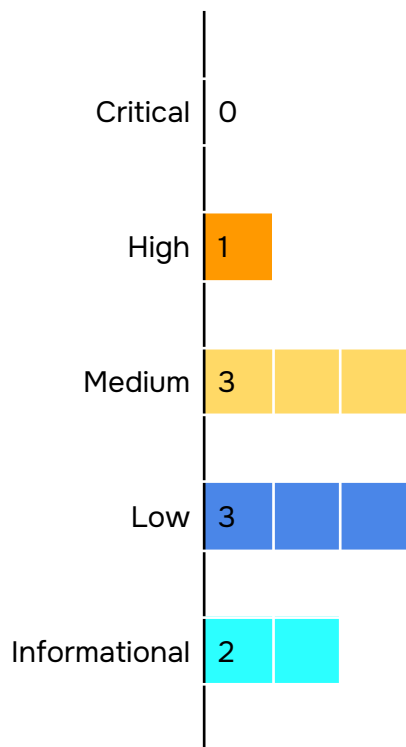
Contracts
src/generated/OrderBookV6.pointers.sol src/generated/RouteProcessorOrderBookV6ArbOrderTaker.pointers.sol src/generated/GenericPoolOrderBookV6ArbOrderTaker.pointers.sol src/generated/OrderBookV6SubParser.pointers.sol src/generated/RouteProcessor4.pointers.sol



```
src/generated/GenericPoolOrderBookV6FlashBorrower.pointers.sol
src/abstract/OrderBookV6ArbTaskGated.sol
src/abstract/OrderBookV6FlashLender.sol
src/abstract/OrderBookV6ArbOrderTaker.sol
src/abstract/OrderBookV6ArbCommon.sol
src/abstract/OrderBookV6FlashBorrower.sol
src/concrete/ob/OrderBookV6.sol
src/concrete/parser/OrderBookV6SubParser.sol
src/concrete/arb/RouteProcessorOrderBookV6ArbOrderTaker.sol
src/concrete/arb/GenericPoolOrderBookV6FlashBorrower.sol
src/concrete/arb/GenericPoolOrderBookV6ArbOrderTaker.sol
src/lib/LibOrderBook.sol
src/lib/deploy/LibRouteProcessor4CreationCode.sol
src/lib/deploy/LibOrderBookDeploy.sol
src/lib/LibGenericPoolExchange.sol
src/lib/LibOrder.sol
src/lib/LibOrderBookSubParser.sol
src/lib/LibOrderBookArb.sol
```



Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

H01 - takeOrders4 delays calculate-phase state persistence, allowing repeated-order batches to bypass stateful order limits

In `takeOrders4()`, each order is evaluated in the main processing loop through `calculateOrderIO()`. That evaluation returns kvs from the calculate entrypoint, but those key-value writes are not persisted immediately. Instead, they are stored on the in-memory `OrderIOCalculationV4` struct and only written later in `handleIO()`, after the main order-processing loop has already finished. The calculation evaluation is performed with an empty `stateOverlay`, so repeated evaluations in the same transaction do not see the calculation-phase writes from prior iterations in that batch. Because `takeOrders4()` does not enforce uniqueness in the `config.orders` array, a taker can submit the same order multiple times, e.g., `[orderA, orderA, orderA]`. Each repeated `calculateOrderIO()` call will observe the same persisted interpreter state, even though earlier iterations in the batch may already have logically “used up” part of a strategy’s intended allowance. Vault balances are still updated immediately, so this does not bypass vault-balance caps, but interpreter-state-based calculate limits are deferred and therefore not visible to later calculations in the same batch.

Path: `src/concrete/ob/OrderBookV6.sol`

Impact: Orders that rely on interpreter state in the calculate entry point to enforce cumulative limits, such as per-block, per-epoch, or rolling execution caps, can be over-consumed in a single `takeOrders4` transaction by simply repeating the same order in the batch. This can allow a taker to bypass stateful strategy limits and drain the order up to the remaining vault balance.

Recommendation: Persist calculate-phase kvs before evaluating the next order in the batch, or pass the prior calculate writes forward through `stateOverlay` so later evaluations see them. If deferred persistence must remain, then enforce uniqueness of order hashes within a single `takeOrders4()` call.

Found in: e686b4d3

Status: New



**** Medium**

M01 - Vault ID 0 on Order Input Side Requires Pre-existing Ambient Balance on OrderBook

Raindex uses `vaultId == 0` as a special mode where settlement happens directly against a user's wallet instead of an internal orderbook vault. In this mode, credits are applied by immediate `pushTokens()` transfers, and debits are applied by immediate `pullTokens()` transfers.

The problem is that `recordVaultIO()` always executes `increaseVaultBalance()` before `decreaseVaultBalance()`. For `vaultId == 0`, this means the contract may try to send tokens out before it has actually received the tokens that should fund that payment.

In `takeOrders4()`: The order pair is fixed (e.g., owner receives token A, gives token B). `recordVaultIO()` runs inside the per-order loop (L521) and pushes token A to the owner. But the taker only pays token A after the loop completes (L569: `pullTokens(msg.sender, orderInputToken, totalTakerOutput)`). The OrderBook must therefore already hold a sufficient balance of token A from unrelated prior deposits by other users.

In `clear3`: Alice's `recordVaultIO()` (L631) executes before Bob's (L632). If Alice uses `vaultId == 0` on her input side, the OrderBook pushes tokens to Alice that should logically come from Bob's output. But Bob's output vault debit has not been processed yet, so the OrderBook must again rely on ambient balance.

Path: `src/concrete/ob/OrderBookV6.sol`

Impact: Orders using `vaultId == 0` on the input side will revert with insufficient balance if the OrderBook contract does not already hold enough of that token from unrelated positions. The trade is logically fundable (taker is willing and able to pay), but the execution order prevents it from completing. Success depends on ambient contract balances from other users' deposits – a fragile cross-position dependency.

Recommendation: Restructure settlement to pull before push:

- in `takeOrders4()`, collect the taker's payment before processing `vault==0` credits;
- in `clear3()`, buffer wallet-settlement transfers until both legs are processed.

Alternatively, documenting that `vaultId == 0` on the order input side (receive-to-wallet) requires the OrderBook to already hold sufficient ambient balance of that token.

Found in: e686b4d3

Status: New



M02 - order-counterparty() is exposed in add/remove post-actions even though that context row is never populated

The add/remove order flows build the post-action calling-context column with only two values: the order hash and the order owner. However, the shared context schema defines a third row in that same column for order-counterparty, and the subparser always exposes the order-counterparty() word as if it were available. In practice, this means the parser surface promises more context than these entrypoints actually provide. This is not merely a documentation mismatch: the interpreter uses checked indexing, so attempting to read the missing third row reverts rather than returning a default zero value.

Path: src/concrete/ob/OrderBookV6.sol, src/lib/LibOrderBook.sol, src/lib/LibOrderBookSubParser.sol

Impact: An expression author can write a syntactically valid post-action for addOrder4() or removeOrder3(), have it parse and deploy successfully, and only discover at runtime that it always reverts when order-counterparty() is used. That creates a real authoring footgun and makes the subparser misleading for these specific entrypoints.

Recommendation: Either populate the third calling-context row during add/remove post-actions, or stop exposing order-counterparty() for these contexts. The cleaner long-term fix is to make context availability explicit per entrypoint rather than presenting one global word surface.

Found in: e686b4d3

Status: New

M03 - arb4 computes flash-loan size from minimumIO without respecting IOIsInput semantics

In the flash-borrower path, arb4() determines the flash-loan amount by converting takeOrders.minimumIO using the borrowed token's decimals. That only makes sense when minimumIO represents the taker input side of the trade. But in takeOrders4(), the meaning of minimumIO is controlled by IOIsInput: if IOIsInput is false, then minimumIO refers to taker output instead. In that configuration, arb4() is effectively treating a limit expressed on one side of the trade as though it were the amount of the borrowed token.

Path: src/abstract/OrderBookV6FlashBorrower.sol, src/concrete/ob/OrderBookV6.sol



Impact: The flash-borrower path does not correctly support the full TakeOrdersConfigV5 surface. Valid order-taking configurations can fail, borrow the wrong amount, or otherwise behave incorrectly when sent through arb4. This is a logic inconsistency between the direct take-order API and the flash-borrower wrapper.

Recommendation: Either explicitly require IOIsInput == true in arb4() and revert otherwise, or compute the flash-loan amount from the actual token side that must be borrowed rather than directly reusing minimumIO.

Found in: e686b4d3

Status: New

* Low

L01 - Rounding policy in pullTokens and pushTokens can trap residual dust in the orderbook

When a float amount must be converted to a token's fixed-decimal representation, pullTokens rounds lossy conversions up, ensuring the contract never receives less than its internal accounting expects. Conversely, pushTokens rounds lossy conversions down, ensuring the contract never sends more than it can safely account for. This is a conservative, protocol-favoring approach and is internally consistent. The tradeoff is that tiny residual amounts can accumulate in the contract over time whenever conversions are lossy. Those leftovers are not assigned to any user vault and do not appear to have an explicit recovery path.

Path: src/concrete/ob/OrderBookV6.sol

Impact: The system can accumulate non-user-owned token dust over time. This is not an exploitable insolvency issue, but it is a real accounting and UX issue, especially for high-volume pairs or unusual decimal combinations.

Recommendation: Keep the conservative rounding if solvency is the priority, but document it explicitly and consider an explicit dust-accounting or sweep mechanism so these balances do not remain permanently orphaned.

Found in: e686b4d3

Status: New



L02 - Deposit and withdraw post-action contexts include token decimals that the subparser does not expose

The deposit and withdraw flows prepare a richer raw post-action context than the author-facing parser surface actually exposes. In `deposit4()`, the post-action context column contains five values: token, vault id, vault balance before, vault balance after, and token decimals. In `withdraw4()`, the column contains six values: token, vault id, vault balance before, vault balance after, target amount, and token decimals. However, the `OrderBook` subparser only defines words for the non-decimal fields. Deposit expressions can read `depositor`, `deposit-token`, `deposit-vault-id`, `deposit-vault-before`, and `deposit-vault-after`, while withdraw expressions can read `withdrawer`, `withdraw-token`, `withdraw-vault-id`, `withdraw-vault-before`, `withdraw-vault-after`, and `withdraw-target-amount`. There are no corresponding `deposit-...-decimals` or `withdraw-...-decimals` words, and the parser tables are sized exactly to exclude them. As a result, the low-level execution context contains data that user-authored Rain expressions cannot address.

Path: `src/concrete/ob/OrderBookV6.sol`, `src/lib/LibOrderBookSubParser.sol`, `src/concrete/parser/OrderBookV6SubParser.sol`

Impact: This does not create an unsafe execution path, but it does create an avoidable mismatch between the raw runtime context and the exposed authoring surface. Expression authors cannot use token decimals in deposit/withdraw post-actions, even though those values are already being computed and passed through execution. It also means some gas and memory are spent carrying data that is effectively unreachable from the supported language surface, increasing the chance that maintainers or integrators assume the deposit/withdraw context is fully exposed when it is not.

Recommendation: Make the raw context and the parser surface consistent. The preferred fix is to add explicit words such as `deposit-token-decimals` and `withdraw-token-decimals`, wire them into the deposit and withdraw parser tables, and include them in the authoring metadata. If exposing decimals is not actually desired, the alternative is to stop appending those unused rows to the post-action context, so the execution context contains only fields that expressions are allowed to access.

Found in: e686b4d3

Status: New

L03 - RouteProcessorOrderBookV6ArbOrderTaker bypasses TOFU decimal handling and is less compatible with non-standard tokens

Most of the protocol resolves token decimals through TOFU helpers such as `LibTOFUTokenDecimals.safeDecimalsForToken`, which are designed to handle



inconsistent or non-standard ERC20 metadata behavior more safely. However, `RouteProcessorOrderBookV6ArbOrderTaker` directly queries decimals via `IERC20Metadata(token).decimals()` when converting float amounts for the route processor.

This makes the route-processor arb path less robust than the rest of the codebase and can cause it to revert for tokens that the wider system is otherwise prepared to handle through TOFU-based decimal resolution.

Path: `src/concrete/arb/RouteProcessorOrderBookV6ArbOrderTaker.sol`

Impact: The `RouteProcessor` arb path handles token decimals less defensively than the rest of the protocol. Tokens that behave inconsistently or unexpectedly around decimals metadata may be rejected or handled differently in this wrapper even when core orderbook flows rely on the protocol's TOFU-based decimal path. This creates avoidable integration inconsistency and makes behavior in the `RouteProcessor` path less predictable than elsewhere in the system.

Recommendation: Replace direct `IERC20Metadata(...).decimals()` calls with the protocol's standard TOFU decimal helpers, such as `LibTOFUTokenDecimals.safeDecimalsForToken`, so this arb path matches the rest of the system's token-handling behavior.

Found in: e686b4d3

Status: New

Informational

I01 - `onTakeOrders2` can be abused to drain accidental balances

The base `onTakeOrders2()` hook is intentionally public and unrestricted. The generic-pool implementation then treats its calldata as a fully caller-controlled (spender, pool, encodedFunctionCall) tuple, gives spender an unlimited allowance on the selected token, forwards the contract's entire ETH balance to pool, executes the external call, and only revokes approval after the call returns. The design comment says this is safe because the contract is expected to hold no funds between operations. That assumption is fragile: if funds do end up on the contract accidentally, this hook provides a direct extraction path.

Path: `src/abstract/OrderBookV6ArbOrderTaker.sol`,
`src/concrete/arb/GenericPoolOrderBookV6ArbOrderTaker.sol`,
`src/lib/LibGenericPoolExchange.sol`



Impact: Any ERC20 or ETH accidentally sent to the generic-pool order taker can be exfiltrated by an arbitrary external caller without using the intended `arb5()` workflow. This is more serious than the general “next arb caller gets stray ETH” footgun because it exposes a direct, arbitrary approve-and-call primitive over accidental balances.

Recommendation: Restrict `onTakeOrders2()` so it can only be called as part of an active orderbook callback, or track arb execution state and reject direct external entry. If statelessness is a core design assumption, the contract should defensively reject unsolicited value where possible rather than relying on “nobody should send funds here.”

Found in: e686b4d3

Status: New

I02 - takeOrders4 Push-Callback-Pull Pattern Exposes Partial State

The execution order in `takeOrders4()` is:

1. `recordVaultIO()` for all matched orders (vault balances updated)
2. `pushTokens()` to taker (output tokens sent – L562)
3. `onTakeOrders2()` callback to taker if data provided (L564-567)
4. `pullTokens()` from taker (input tokens received – L569)
5. `handleIO()` for all matched orders (L572-575)

During step 3, the taker's callback observes a state where:

- Vault balances are already updated (all orders recorded)
- Output tokens have been sent to the taker
- Input tokens have NOT been received yet
- `vaultBalance2()` returns updated values
- The contract's actual token balance is depleted

Path: `src/concrete/ob/OrderBookV6.sol`.

Impact: No direct exploit due to `nonReentrant` on the `OrderBook`. However, integrators making external calls during the callback may face unexpected state if they depend on the `OrderBook`'s actual token balance matching internal accounting.

Recommendation: Document the exact contract state during the `onTakeOrders2()` callback for integrators. Specifically note that `maxFlashLoan()` will return a reduced value during the callback.

Found in: e686b4d3

Status: New



General Risks and Assumptions

- **Numerical Approximation Variance:** Advanced operations, including logarithms (log10), exponentiation (pow), and square roots (sqrt), rely on on-chain lookup tables and linear interpolation. Because these are numerical approximations rather than exact analytical solutions, high-precision round-trip calculations may exhibit minor deviations.
- **Precision Loss and Truncation:** Although the system avoids binary floating-point errors, it remains subject to finite precision. Operations resulting in repeating decimals or requiring rescaling to match specific token decimal configurations (via TOFU integration) will undergo rounding towards zero. This can lead to minor precision loss that may compound over long execution sequences.
- **Heightened Gas Consumption:** The implementation of complex financial logic and custom floating-point math in Solidity and Yul is significantly more gas-intensive than native integer arithmetic. Operations involving iterative algorithms or table lookups may result in substantial gas overhead, particularly in high-frequency trading scenarios or within large execution loops.
- **Deterministic Fail-Fast Behavior:** The protocol enforces a strict error-handling model that reverts on invalid mathematical states (e.g., division by zero or exponent overflow) instead of propagating invalid states like NaN or Infinity. While this ensures system integrity, it requires that interacting systems and automated keepers perform robust pre-flight validation to avoid transaction failures and wasted gas.
- **Non-Standard and Non-Static ERC20 Assets:** The project assumes plain ERC20 behaviour: transferred amounts match posted amounts and balances are stable for the duration of a settlement path. Fee-on-transfer, rebasing, balance-indexed, and similar token designs break that assumption. Project owners and users should expect incorrect accounting, failed trades, or unfair outcomes if such assets are listed without explicit policy. Supporting them is not a default guarantee; scope, listing rules, and user communication need a deliberate decision.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.

